1 2 3 4 5 6 7 8 9 10
11 12 13 14 15 16 17 18 19 20

→ conda create -p venv python==3.10

(*) Environment is necessary in data science projects because new packages are released every few months, so environments help us keep things organised.

→ Step (1) → create env.
$ conda create -p venv python==3.12

(2) → Activate it.
$ conda activate venv/

Python -

→ Case sensitive language.

eg: identifiers are case sensitive
Vinek ≠ vinek

\ → backslash character → used to escape spaces or lines.

eg : a = 7+3+4\
2+1 → or to print special characters

⇒ a = 7+3+4+2+1

Multiple statements → a=18 ; c=5 ; z=a+c

Type → type( ) → returns type of identifier.

Identifiers → can only start with letter or underscore (_)
→ can contain numbers
→ case sensitive

Types — 25 — int
61 — float
'abc' — str
True → boolean

Convert type : a = 25 | Type int
a = str(a) | string.
a = int(str(a)) | int.

Note → str(), int() returns a value, doesn't update it

Type conversions

	Type	
a = 5.11	float	
a = ~~sta~~ int (a)	int	→ a = 5.
a = float(a)	float	→ a = 5.0

Note → if we don't reassign, the type don't change.

eg.

a = 5.11
~~print (type(a))~~
~~⇒ < class 'float' >~~

a = 5.11
print (type (a))

⇒ < class 'float' >

print (str (a))
⇒ 5.11

~~print~~ type (a)
⇒ $\boxed{\text{float}}$

Input function

input ( )

⇒ Always takes string input by default.

bool ( ) → returns a True / False

eg. c = bool(3+2)

⇒ c = True.

Operators in Python

+ → addition

− → subtraction

* → multiplication

/ → division

// → floor division → absolute quotient.
(or aka rounding down)

% → remainder after division (modulus)

** → exponential operation (raise to the power of)

Comparison → returns bool.

== → is equal to ?
~~→ Comparison~~ → returns bool.

!= → is not equal to ?

Comparison

> → greater than

< → less than

>= → greater than, equal to

<= → less than, equal to

Logical

1(0)

Logical operators

AND → returns True if both items are True, else False.

NOT → returns the opposite/inverse of the given boolean variable

OR → returns True if any 1 of the items is True, else False.

Factorial → use recursive function/algo.

```
def fact(n):
    if n==0:
        return 1
    else:
        return n* fact(n-1)
```

⇒ gives the factorial of a number

Swapping in one line.

a = 5 ; b = 10

$$\boxed{a, b = b, a}$$

OR

$$\boxed{a = (a + b) - (b = a)}$$

Slicing

(*) Reversal techniques ~~etc~~

[::-1]

inclusive — excluded — skips (-ve in reverse)

=> [start : stop : step]

eg : mlist = [10, 20, 30, 40, 50, 60, 70]

Description	Code	Result
First 3 elements	mlist[:3]	[10, 20, 30]
Elements index 2 – 5	m[2:6]	[30, 40, 50, 60]
Every other elem	m[::2]	~~[20, 40, 60]~~ ✗ [10, 30, 50, 70]
Reverse the list	m[::-1]	[70, 60, 50, 40 ...]
Last 2 elements	m[-2:]	[60, 70]